GOLD: Flipped Friday #2 (Fri Sep 25)

Flipped Friday #2

How this hour works

Midterm 1.

- Midterm 1 is **Wednesday, September 30**. This week's material (writing classes) is **not** on it.
- You have already received an email with your room and time assignment.
- You must take the midterm on the lab computers.
- Practice test available on Ed

Participation

- **iClicker:** participation is based on the polls only.
- **Ed challenges**: the test case scores are just for practice; we never include these in the class participation.

Writing your first classes

This week you watched classes get written. Today's goals are to repair a broken class and write one from scratch, the way it might look on an exam question. UTA staff are in the aisles.

- Warm-up poll
- Activity 1: Fix the Wheel (~10 min): three planted bugs
- Whole-room: two polls
- Activity 2: A Cup of Dice (~15 min): a class that holds objects
- Close + last poll.
- *Finished early? Try Deck, the optional last slide.*

Two files this time:

Each challenge is a small workspace: the class you edit, plus a `Main.java` that already uses it.

Run runs `Main`; **Test** runs the tests.

Warm-up

Warm-up

Pause and predict

Q. `Wheel` has exactly one constructor. What happens on the line `Wheel w = new Wheel();`?

```
class Main {
    void main() {
        Wheel w = new Wheel();
        IO.println(w.getRadius());
    }
}

class Wheel {
    private double radius;
    private String material;

    Wheel(double radius, String material) {
        this.radius = radius;
        this.material = material;
    }

    double getRadius() {
        return radius;
    }
}
```

- **A.** Runs and prints `0.0`
- **B.** Runs and prints `null`
- **C.** Compile error
- **D.** Runs, then crashes inside `getRadius()`

Friday 2: Fix the Wheel

This `Wheel` class compiles, and one thing about it already works. Three things do not.

Each fix is one or two lines using an idea from this week.

Bug 1. `new Wheel(27.5, "rubber")` reports a radius of `0.0`.

Bug 2. Three wheels exist in `Main.java` and the count says `0`. Then four radius readings push it to `4`. The count is counting the wrong event. Where does "count every wheel" belong?

Step 3. At the bottom of the file, remove the `//` from the `describe()` method and press **Run**. It will not compile. Read what the compiler says before you change anything. Then fix it.

Two files, as in the take-homes: `Wheel.java` is the one you edit; `Main.java` uses it.

Run runs `Main`. Press **Test** to run the tests.

Whole-Room: the Modal Wrong Answer

Whole-room: the modal wrong answer

```
class Main {
    void main() {
        Wheel a = new Wheel(27.5, "rubber");
        Wheel b = new Wheel(5.0, "wood");
        Wheel c = new Wheel(13.0, "steel");
        IO.println(Wheel.getWheelCount());   // three wheels exist
        IO.println(a.getRadius());
        IO.println(b.getRadius());
        IO.println(Wheel.getWheelCount());   // no new wheels since
    }
}

class Wheel {
    private double radius;
    private String material;
    private static int wheelCount = 0;

    Wheel(double radius, String material) {
        radius = radius;
        this.material = material;
    }

    double getRadius() {
        wheelCount++;
        return radius;
    }

    static int getWheelCount() { return wheelCount; }
}
```

Pause and predict (Q1)

Q. With this constructor, `new Wheel(27.5, "rubber").getRadius()` returns…

- **A.** `27.5`
- **B.** `0.0`

- **C.** Compile error: `variable radius is already defined`
- **D.** `null`

Then the count. Three wheels exist and the first line printed `0`; two radius readings later it says `2`.

Pause and predict (Q2)

Q. Three wheels exist and `getWheelCount()` says `0`; after two radius readings it says `2`.

Where does `wheelCount++` belong?

- **A.** In `getWheelCount()`, so the count is fresh each time it is asked for
- **B.** In the constructor, which every new `Wheel` runs exactly once
- **C.** In `main`, after each `new Wheel(...)`
- **D.** In every getter, so that each use of a wheel counts

Friday 2: A Cup of Dice

`Die.java` is given to you from this week's lectures. We added the overloaded constructor `new Die()` meaning six-sided. Do not change `Die.java`.

Write **`DiceCup`**, a class that holds up to a fixed number of dice:

- **Fields.** A `Die[]` for the dice (its length is the cup's *capacity*) and an `int count` for how many dice are really in the cup. Both are already declared for you.

Implement the following methods in `DiceCup.java`:

- **`DiceCup(int capacity)`**: makes the array and starts the count at 0. Remember what a new array of objects holds: every slot is `null`. No dice exist yet.
- **`addDie(Die d)`**: puts `d` in the next free slot and returns `true`. If the cup is already full, changes nothing and returns `false`.
- **`getCount()`**: how many dice are in the cup.
- **`rollAll()`**: rolls every die in the cup and returns the total. An empty cup totals 0.

The capacity is how many the cup *could* hold. The count is how many it *does* hold.

Run runs `Main`. Press **Test** to run the tests; one of them puts two dice in a cup of three.

Closing

Closing

Pause and predict

Q. A cup of 3 holds 2 dice, and `rollAll()` is written like this. What happens?

```
public int rollAll() {
    int total = 0;
    for (int i = 0; i < dice.length; i++) {
        total += dice[i].roll();
    }
    return total;
}
```

- **A.** Returns the total of the two dice
- **B.** Returns the total of three rolls; an empty slot rolls `0`
- **C.** Throws `NullPointerException`
- **D.** Compile error: `length` is not a method call

Friday 2 (optional): Deck

Optional, for anyone who finishes the cup of dice.

`Card.java` is given: Wednesday's immutable card with a `toString`. Do not change it.

Write **`Deck`**: 52 cards in a `Card[]`, plus a `count` of how many have not been dealt yet. This is the cup of dice again: the array is the capacity, `count` is what is really there.

- **`Deck()`**: makes the 52-slot array and fills it with a nested loop over two arrays you write, one of suits and one of ranks (`"Ace"`, `"2"` … `"10"`, `"Jack"`, `"Queen"`, `"King"`). Suits outer, ranks inner.
- **`cardsLeft()`**: how many cards remain.
- **`deal()`**: takes the top card, from the **end** of the filled part, and returns it. An empty deck returns **`null`** and must never crash.

Run runs `Main`. Press **Test** to run the tests.